

PRACTICAL-7

AIM:- Implementation of making a change problem using dynamic programming.

Making a change problem:

Algorithm:

Algorithm:
Input: no. of coins n and denominations $\text{coins}[0..n-1]$
Amount to be changed amount.
Output: minimum no. of coins required to make the given amount

step 1: start
step 2: Read the no. of coins n .
step 3: Read the coin denominations.
step 4: Read the required amount.
step 5: Create a DP array $\text{dp}[\text{amount}+1]$
step 6: Set: $\text{dp}[0] = 0$
step 7: for each amount i from 1 to amount
• set $\text{dp}[i] = 32767$
• check every coin denomination coin
• if $\text{coins}[j] = i$, calculate:
$$\text{dp}[i] = \min(\text{dp}[i], \text{dp}[i - \text{coins}[j]] + 1)$$

step 8: After filling the DP table, $\text{dp}[\text{amount}]$ contains the minimum no. of coins
step 9: Display $\text{dp}[\text{amount}]$
step 10: stop.

Program:

```
#include <iostream>

#include <algorithm>

using namespace std;

int main()
{
    int n, amount;

    cout << "Enter number of coins: ";
    cin >> n;

    int coin[n];

    cout << "Enter coin denominations: ";
    for (int i = 0; i < n; i++)
    {
        cin >> coin[i];
    }

    cout << "Enter amount: ";
    cin >> amount;

    int dp[amount + 1];

    dp[0] = 0;
```

```
for (int i = 1; i <= amount; i++)  
{  
    dp[i] = 32767;  
  
    for (int j = 0; j < n; j++)  
    {  
        if (coin[j] <= i)  
        {  
            dp[i] = min(dp[i], dp[i - coin[j]] + 1);  
        }  
    }  
}  
  
cout << "Minimum number of coins required = "  
    << dp[amount] << endl;  
  
return 0;  
}
```

Output:



Output

```
Enter number of coins: 3
Enter coin denominations: 4 5 6
Enter amount: 8
Minimum number of coins required = 2
```

=== Code Execution Successful ===

Time complexity:

Best case:

Time complexity:

Best Case:

for($i = 1$; $i \leq \text{amount}$; $i++$) $\rightarrow$ amount-time

for($j = 0$; $j < n$; $j++$) $\rightarrow n$

if($\text{coin}[j] \leq i$) $\rightarrow 1$

$\text{dp}[i] = \text{minc}$ $\rightarrow 1$

$T(n) = \text{amount} \times n$

$= O(\text{amount} \times n)$

Average case:

Average case:
for ($i=1$; $i \leq \text{amount}$; $i++$) $\rightarrow \text{amount}$
for ($j=0$; $j < n$; $j++$) $\rightarrow n$
if ($\text{corn}[j] \leq 1$) $\rightarrow 1$
dp [i] = min $\rightarrow 1$
 $T(n) = \text{amount} \times n$
 $T(n) = O(\text{amount} \times n)$

Worst case:

Worst Case:
for($i=1$; $i \leq \text{amount}$; $i++$) $\rightarrow \text{amount}$
for($j=0$; $j < n$; $j++$) $\rightarrow n$
if($\text{coin}[j] \leq i$) $\rightarrow 1$
 $\text{dp}[i] = \min()$ $\rightarrow 1$
 $T(n) \approx \text{amount} \times n$
 $T(n) \approx O(\text{amount} \times n)$

Conclusion:

The implementation and time analysis of the Making Change Problem using Dynamic Programming helps us understand its efficiency. The algorithm builds a DP table and considers different coin denominations to find the minimum number of coins required to make the given amount. It has a time complexity of $O(n \times \text{amount})$ in the best, average, and worst cases. Therefore, Dynamic Programming provides an efficient and systematic solution for solving the Making Change Problem.